

Assessing the state of software in a large enterprise

Randy L. Hackbarth · Audris Mockus ·
John Douglas Palframan · David M. Weiss

Published online: 7 November 2009
© Springer Science + Business Media, LLC 2009

Abstract To be relevant to the goals of an enterprise, an industrial software engineering research organization must identify problems of interest to, and find solutions that have an impact on, the software development organizations within the company. Using a systematic measurement program both to identify the problems and assess the impact of solutions is key to satisfying this need. Avaya has had such a program in place for about seven years. Every year we produce an annual report known as the State of Software in Avaya that describes software development trends throughout the company and that contains prioritized recommendations for improving Avaya's software development capabilities. We start by identifying the goals of the enterprise and use the goal-question-metric approach to identify the measures to compute. The result is insight into the enterprise's problems in software development, recommendations for improving the development process, and problems that require research to solve. We will illustrate the process with examples from the Software Technology Research Department in Avaya Labs whose purpose is to improve the state of software development and know it. "Know it" means that improvement should be subjectively evident and objectively quantifiable. "Know it" also means that one must be skilled at identifying the data sources, performing the appropriate analyses to answer the questions of interest, and validating that the data are accurate and appropriate for the purpose. Examples will include how and why we developed a measure of software quality that appeals to customers, how and why we are studying the effectiveness of distributed software development, and how and why we are helping development organizations to adopt iterative development methods. We will also discuss how we keep the company and the department apprised of the current strengths and weaknesses of software development in Avaya through the publication of the annual State

R. L. Hackbarth
Research, Avaya Labs, Western Springs, IL 60558, USA
e-mail: randyh@avaya.com

A. Mockus · J. D. Palframan (✉)
Research, Avaya Labs, Basking Ridge, NJ 07920-2311, USA
e-mail: palframan@avaya.com

A. Mockus
e-mail: audris@avaya.com

D. M. Weiss
Department of Computer Science, Iowa State University, Ames, Iowa 50011, USA
e-mail: weiss@cs.iastate.edu

of Software in Avaya Report. Our purpose is both to provide a model for assessment that others may emulate, based on seven years of experience, and to spotlight analyses and conclusions that we feel are common to software development today.

Keywords Software development · Software engineering · Software quality · Software process improvement · Software practices · Software assessments · Software metrics · R&D assessment · Distributed and offshore development · Iterative development · Code growth · R&D domain expertise · R&D productivity · Agile Development

1 Introduction

This paper describes a continuing project to assess the software development capabilities of a large telecommunications company whose existence depends on those capabilities. Avaya is a provider of enterprise-based telecommunications systems and develops large, complex, real-time software systems that are either embedded or are standalone products. Its R&D organization, which numbers over 2000, is continually called upon to improve the quality of its software, to decrease its time-to-market, and to decrease the cost of development and maintenance of its software. Under these pressures it is critical to identify changes in development processes, environments, cultures, and tools that maximize improvement, that can be accomplished with existing resources, and that produce measurable results.

The Avaya Resource Center for Software Technology (ARC) produces an annual report known as the State of Software in Avaya that describes software development trends throughout the company and that contains prioritized recommendations for improving Avaya's software development capabilities. Priorities are assigned to the recommendations based on their expected impact and on the estimated capability of the software development organizations to implement them. A primary goal of the ARC is to improve the state of software in Avaya and to know it. "Knowing it" means that improvement should be subjectively evident and objectively quantifiable. Accordingly, part of the report is devoted to showing year over year changes in Avaya's development capabilities, with attention paid to the impact of previous recommendations. The ARC has been producing the annual State of Software in Avaya report for seven years. We hope that our methods for assessing trends may be a model for others to use.

This paper discusses the methods used in and the lessons learned from producing the annual report, using examples taken from the reports. Section 2 describes the approach used to create the report and Section 3 describes the data used as input to produce the report and methods for its collection. Section 4 describes some of the analyses performed on the data and some sample conclusions drawn from those analyses. Section 5 describes our assessment approach for a set of software development practices. Section 6 illustrates the types of recommendations provided in the report, how the recommendations are deployed, and their impact. Section 7 discusses what the ARC has learned from producing the reports, how the report continues to evolve, how we expect it to evolve in the future, and the applicability to other organizations of the practices the ARC uses in crafting the report.

2 Assessment Approach and Mechanisms

As described in previous work, we use a goal-oriented approach for the assessment of Avaya's software development capabilities (Weiss et al. 2002), employing the goal-question-metric paradigm (Basili and Weiss 1985, 1984; van Solingen and Berghout 1999).

We do not attempt to characterize software development with a single number, such as with the CMMI approach. Rather, we first establish the goals of software development in Avaya based on Avaya's business goals and on the goals established by the development organizations in order for the business goals to be achieved. As noted by Rifkin (2001), different organizations adopt different product development styles depending on their approaches to their businesses. Companies that most highly prize innovation tend to have a different style than companies that most highly prize quality. We believe that it is a mistake to evaluate the different styles with the same set of measures.

In addition, comparing our data and results with other enterprises is problematic because of the difficulty in assuring comparability among data. Validating our data internally is a difficult task, as we discuss in Section 3.2, and as the reader may infer from our discussions of the types of data that we analyze in our examples in Section 4. As an example, simply trying to determine how much code Avaya has in its code bases is complicated by a number of factors, including, but not limited to, the following.

1. What source code languages to consider. Avaya's products use C, C++, C#, Java, and many others, in very unequal parts.
2. Whether to include third party code or not, and how to identify it in code bases in either case.
3. How to identify duplicated code that occurs when a new configuration control system goes into use in a product and existing code is copied into it.

Comparing code size with other enterprises would require consistent answers to these questions from all enterprises involved in the comparison.

As a result, except for a few, very specific cases, we do not try to make comparisons between our data and other enterprises. Even assuming that we could be assured of comparability in the data, we would then have to take into consideration whether or not the enterprises had the same goals. For example, comparing product development time for a company whose goal is rapid time-to-market with a company whose goal is innovation or high quality may lead to misleading results and actions, particularly if one is not aware of the difference in goals. Put another way, we do not benchmark our software measures against other enterprises, either in an attempt to do better or to relax with the thought that we are better. Rather, we compare against our own goals and try to improve with respect to them.

In planning our approach, we have been particularly influenced by Walston and Felix's early work (Walston and Felix 1977), by work on the Experience Factory by Basili et al. (1992), and by the foundational work on industrial software measurement by Grady and Caswell (1987). Like Quality Function Design (QFD) (Akao 2009), we analyze quantitative data as well as seek out and prioritize spoken and unspoken customer (Avaya R&D) needs using a list of questions. The product we generate is a series of recommendations with deliverable actions that we expect will improve Avaya's software development practices. Unlike QFD, we do this on a global scale, and quality is only one of the aspects we seek to optimize in R&D.

Since Avaya produces a number of different products for different markets and market segments, the business goals of the individual product development organizations differ, although all strive to meet certain common goals. As an example, in recent years improvement in customer satisfaction ratings has been a prominent goal. All divisions are striving to meet this goal. On the other hand, in certain markets, low cost and ease of installation and use are primary goals, whereas in others very high reliability is the dominant goal. Both common and individual goals color our assessments and the recommendations we make to different divisions.

Data sources for the report include both quantitative and qualitative analysis of data. Quantitative data comes from sources such as distributions of defects found by customers,

modifications made by software developers and described in modification reports (MRs), code in Avaya's software repositories, and demographic data. Qualitative data results from sources such as interviews with software developers, software managers, software product managers, and others¹, impressions and data gained from participating in architecture and other reviews, and from specialized assessments of particular issues, such as how well Avaya development organizations are applying iterative development techniques. The introduction to the 2008 State of Software in Avaya Report describes the sources of information used in producing the report. It includes the following description (slightly edited to preserve confidentiality).

The State of Software in Avaya report series is published by the Avaya Resource Center for Software Technology (ARC) to provide periodic snapshots of Avaya's software production capability. Our goal is to give Avaya R&D organizations a picture of where they need to focus resources to make improvements to achieve operational excellence. Our intent is to give the reader insight into how well Avaya is using its software production capacity, including what resources are available for software development, how effective those resources are, where they are located, and whether they are sufficient.

The reports draw on learning from the services and analyses that the ARC provides to Avaya projects². This year's report draws on the following sources.

- *Our learning from our work with the divisions on a software improvement initiative based on last year's report.*
- *Our 2008 assessment of iterative development deployment in Avaya R&D*
- *Our coordination of the 2007 Avaya Software Symposium and the 2008 Avaya Test Forum.*
- *Our participation in a variety of software architecture reviews, and other services.*
- *Our conduct of individual and small group input sessions with more than 120 members of the R&D community, and with program and product management from all divisions, distributed across Avaya's worldwide R&D locations. Our findings have been reviewed with and adapted based on feedback from these individuals.*
- *Our quantitative analyses of data such as demographic data obtained from SAP and other sources, customer found defects, code in Avaya software repositories, project data, and MR data reported in the data warehouse and various configuration management systems used in Avaya.*

The [Appendix](#) contains a list of example questions used in the interviews to which the Introduction refers.

3 Data Sources

We primarily rely on the increasingly central administration of key software development support tools such as version control, problem tracking systems, and service support systems to provide our input data. Employee directories provide additional means to capture statistics for the entire enterprise. The alternative, to request each project to provide detailed metrics on a variety of their dimensions, is not likely to result in a uniform report across projects and is unlikely to include all projects

¹ In 2008 we interviewed 120 people from R&D and Product Management explicitly for the report.

² A project is undertaken to develop one or more releases of a product. In some cases the development of a release is organized into multiple projects.

To provide context and an overview to readers, we report the amount of code, the productivity of developers, and the quality of the products. This overview is particularly useful for new executives and managers. Because our goals include assessing the impact of our recommendations and detecting potential issues, we focus on obtaining trends of measures over time. This also helps in cases where comparison of results is best done only within a single product or organization. For example, we compare the incidence of customer found defects (CFDs) across successive releases of a single product rather than across different products. The measures we use allow answers to a number of specific practical questions, as indicated and described in Section 4. Because of the amount of data and the sensitivity of some of it, we only provide here a few examples of the data and our analyses.

3.1 Types of Data Analyzed

As previously noted, our quantitative data sources tend to focus on the process by which the software is developed, the demographics surrounding development, and the quality as seen by the developers and by the customers. CFDs and MRs are primary data sources and from them we can extract information about a variety of issues, including types of errors that occur and practices that are used. For example, by examining when an error is found we can usually determine the detection technique used, such as a code review or a system test. We do not spend time analyzing the characteristics of code, either by static measures such as complexity or run-time measures such as performance. We feel that software development proceeds by making changes to code and the best way to view the effectiveness of the process is by analyzing changes to the software. It is also difficult to spot corporate-wide trends by looking at code measures since languages, platforms, and development tools vary across the company; we leave such analysis to the individual projects.

3.2 Data Accuracy

Assuring and estimating the accuracy of our data is a major concern. We deal with large data sets that are distributed across a variety of repositories, where some of the data are entered manually, often by people far removed from the analysis of the data. For example, MR data are stored in different configuration control repositories and the descriptions of the changes in the MRs are entered by software developers or by support teams that may not be aware that their entries become part of a company-wide analysis. Human error or even just vagueness in such descriptions can be a major source of inaccuracy, as discussed and shown in (Basili and Weiss 1984). The problem is compounded when performing analyses that include several different sources or types of data.

There are several distinct dimensions of issues we encounter when trying to assure data accuracy.

1. Precise operationalization of the measures of interest. For example, if we want to report the number of problems a particular release of a product has, we often need to determine the origin of problems to exclude, for example, non-customer-reported problems. In some cases, we need to distinguish and exclude enhancement requests from customers as well. In addition, the number of problems increases over time as more customers use the product for a longer period. If we need to compare past releases to a present release what time interval do we need to choose to count problems for past releases to make a comparison?
2. Errors in reporting. For example, the origin of the problem may be misspecified or it may be filed against the wrong release.

3. Vagueness, ambiguity, and a variety of representations of the same fact associated with manually entered data. For example, the same release might be specified as 3.2, R3.2.0, or r3.2.x.
4. Missing values. For example, a certain number of customer problems may be missing from the repository because some customers are serviced by a third party that may filter only a subset of their customer's problems to Avaya. In particular, international locations are more likely to be serviced by a third party, and, therefore, are likely to have more of their problems missing.
5. Cross-referencing. The most common and most complicated problem occurs when all the problems mentioned in the preceding are compounded by the need to cross-reference data across multiple data sources as described in, for example, (Mockus and Weiss 2008), which describes techniques used to validate data in such complex scenarios.

We believe it is incumbent on us to estimate the error in the analyses that we do, and we continue to seek good methods for doing so. This is a research topic that the measurement community should address. In the following sections, where we present some examples of our analyses, we will provide error estimates where we can reasonably make them.

4 Examples of Analyses

Each year the report analyzes demographic trends, domain expertise, code trends, software quality trends, productivity, predictability, and software practice trends. Each report also focuses on some special topic of interest for the year. For example, in 2008 we analyzed the deployment of architecture guided iterative development in Avaya. Each analysis is based on a study of some set of input data, including cleaning and validating the data. As previously noted, part of the validation process is showing the analysis results to people in a variety of roles in R&D and other organizations and noting their objections and confirmations. We then make a final pass through the data and analyses, making adjustments as necessary. The analyses that follow are each intended to answer some specific question(s), which are noted at the beginning of the section.

4.1 Demographic Analyses

Questions: How is staff distributed across locations and divisions?

How much staff churn is there?

The reports look at the distribution of the current R&D population by location, titles, experience, division, company of origin, and by type of staff (employee, contractor, outsource, or offshore). Understanding the distribution and composition of the R&D organizations helps us understand problems R&D is facing and to spot trends. For example, Fig. 1 shows that many divisions in Avaya are spread across many locations, prompting us to study issues projects have with distributed development. Figure 2 shows a major buildup in offshoring and in outsourcing, and so we are drawn to address issues such as knowledge transfer and how work is partitioned.

Analyzing the origins of the company's current employees provides insight into products, technologies, and markets with which they are familiar. This has impact on software development methodologies and on the locations of experts in various disciplines. A more detailed view (not shown here) reveals for each year hired how many employees remain and how many of the original set have left the company. This can be used to

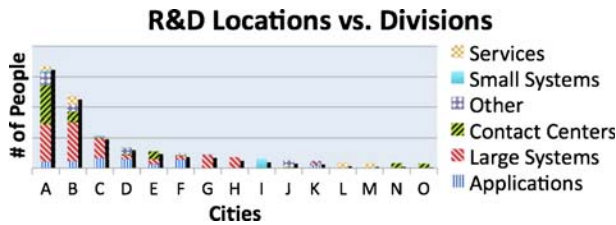


Fig. 1 Distribution of divisions across locations

examine retention policies and morale issues. (This may only be possible because Avaya is a relatively new company and we can access records back to its origin nine years ago.)

The demographic trends can be combined with other trends, such as code growth, to show where support may need to be rebalanced.

Other analyses include staff churn trends, i.e., the turnover rate at different locations, which allows us to estimate the rate of loss (or gain) of domain expertise. Because real-time telecommunications products are quite complex, it can take 6 to 12 months or more before a developer new to the product or company can be effective. The churn trend can be used in analyzing productivity losses caused by staff loss or by training rates for new staff.

Our demographic analyses are based on our corporate personnel data repositories.

In summary, the demographic analyses provide a background and context that assist us in understanding other trends in the company.

4.2 Code Growth

Question: What is the size of Avaya's codebase and how rapidly is it changing?

While each release of every product is built from a set of specific versions of source code files, with each release using specific versions of compile and build tools, no individual division pays attention to the historic growth of the entire code base of all products. The task appears to be straightforward. Let $S(f, t)$ be the size of the file f after the last commit (excluding commits that happened after time t). Then the size of the

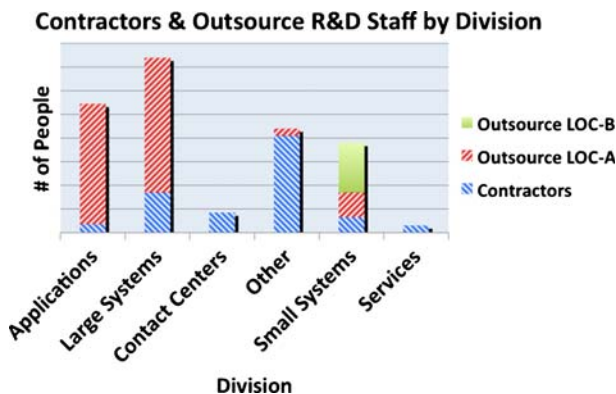


Fig. 2 Outsourcing by division

codebase at time t can be obtained by adding $S(f,t)$ over all files in all version control systems.

However, applying this algorithm is not straightforward. Several questions need to be answered to do so.

1. What files should be considered as the source code?
2. What files represent Avaya's code and what files represent code licensed from some third party or from open source projects?
3. What files are no longer used in Avaya's products?
4. How one does one count files that have been copied from one version control repository to another repository or to another project?
5. How to calculate the size of the files?

We chose Non-Commentary Source Code Lines (NCSL) as the measure of size as calculated by the counting tool that was most widely used in the Avaya projects. (While different line counting tools produce slightly different numbers for NCSL, the variations tend not to exceed 5%.) For code growth analysis, we have focused on the dominant languages used in Avaya, namely C, C++, C# and Java. The determination of copied files was particularly salient to us because many projects switched to a different version control system over the considered period. Some older projects copied their entire codebase to a new version control repository with each major release. Including both old and new version control systems in the analysis could double the size of the codebase for the project. Other projects would copy the source code tree for each new hardware platform that was introduced. The average file size was 125 NCSL, though file sizes were skewed with most files being smaller and very few being very large. The average file size has not changed substantially over time.

Even for projects that stayed within a single version control system the question remains of how to count the distinct branches of the same file. Our approach is to define unique files, i.e., files that are independent of version control systems or branches, and add the sizes of these unique files. To identify unique files we use file copy detection techniques described in (Chang and Mockus 2008). Such file copy detection techniques vary from 70 to 90% in accuracy. We then used the same techniques to identify the unique files that are also present in a very large sample of open source code files (Mockus 2007) to identify code with open-source ancestry.

Third party code in Avaya is usually located within the source code tree in separate directories that are named as third party. Also, third party code is often committed in a single batch, usually, by a person administering the version control system, and is not modified later. The accuracy of the descriptions that indicate that code is third party is reliable, but checked-in-but-not-modified code heuristics may cover some actual product code that was either recently checked in, or did not require any corrections or enhancements.

Finally, to identify the active code base we have looked only at the files modified within the last three years.

We have tracked the code size for Avaya projects dating back to 1996, and have trend graphs for the code base of most Avaya projects. Figure 3 shows the growth of open source and third party code in representative C-like source code files for sixteen major Avaya projects. In 2008, open source code was 29% of the code base for these projects, indicating its importance to commercial software, and third party code was 5% of the code base.

Our source for code growth data is a large sample of Avaya project source code management systems.

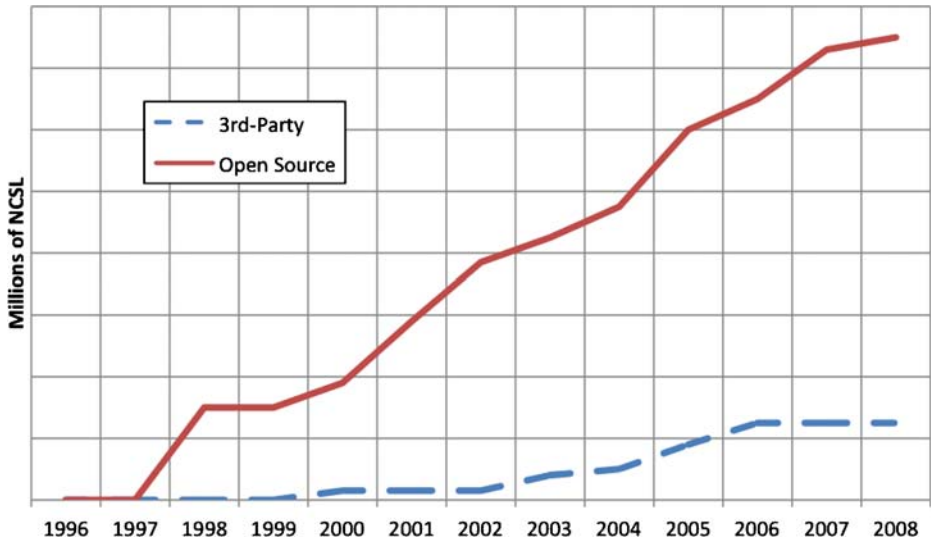


Fig. 3 Growth of representative Avaya code base for C , C++ and Java code files

4.3 Customer Perceived Software Quality

Question: What is the quality of products from the customer viewpoint?

As was shown in (Mockus and Weiss 2008), what customers see is often quite different from in-process measures of software quality such as defect density. Interval Quality (IQ) represents the probability that a customer will observe a failure within a certain interval after software release. The difference between defect density measures and IQ are primarily driven by the software deployment practices. In particular, larger major releases are deployed more gradually, affecting fewer customers and, therefore, raising relatively fewer issues than would be expected based on release size alone. Minor releases are deployed more rapidly, affecting larger numbers of customers, and, therefore, raise more issues than would be expected based on their small size. In Fig. 4 major and minor releases are distinguished by a naming convention of R<major>.<minor> numbering.

We estimate the probability that a customer observes a failure within one month of installation by computing the fraction of customers that had an issue within one month of installation. To calculate the probability we use the population of all customers who had a particular release installed or upgraded one to seven months after the general availability (GA) date, i.e., the date on which the release becomes available to all customers. Customers that install and downgrade a release within a month are counted.

We chose the seven month period to be able to calculate the 3-month probability for the latest release we consider. Another reason was that a new release often appears within seven months. As an example, a customer who has installed Release N of a system seven months after GA would normally contribute to the six month interval for Release N at 13 months after GA. However, if the customer upgraded to Release N+1 prior to 13 months after GA, they would not be included in the six month interval for Release N, and would be counted in the intervals for Release N+1. Finally, the failure rate drops as we move further from the GA date, making it less likely that an unacceptably high failure rate would be observed. Avaya products are critical to customer's ability to communicate, so are up all the time with

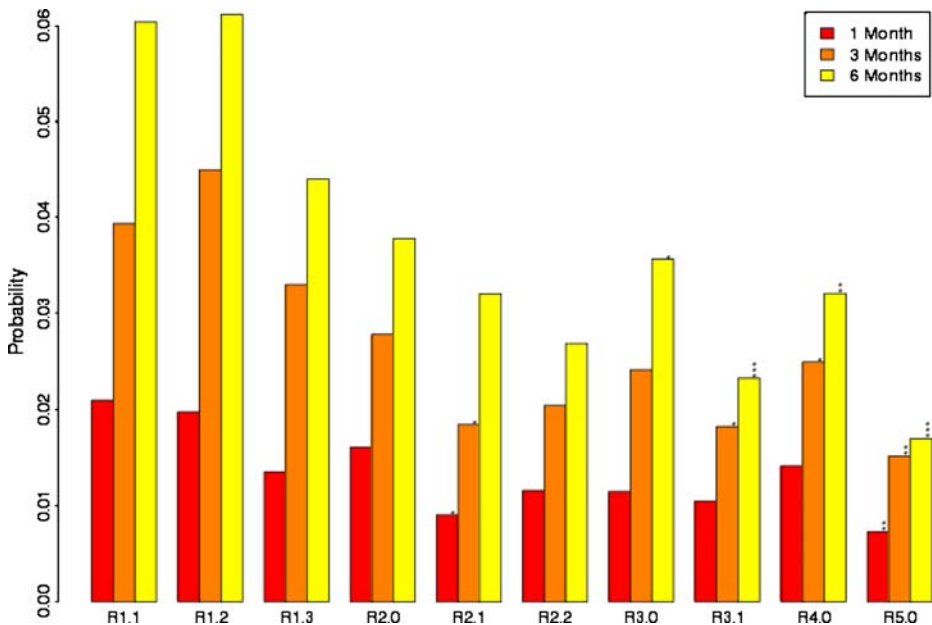


Fig. 4 Interval quality metric trend for a single platform of a single product

reboots only for infrequent maintenance. This characteristic allows us to simplify the charts by using calendar time.

When we do the calculation for the three- and six- month intervals, the populations of systems decline as some systems are upgraded to newer releases during this period and, therefore, do not remain on the same release long enough to be included in the sample. As an example, a customer who installed release N 7 months after GA would normally contribute to the 6 month interval at 13 months after GA. However, if they upgraded to release N+1 prior to 13 months after GA, they would not be included in the six month interval for release N, and would be counted in the intervals for release N+1.

Figure 4 shows the IQ measures for one OS platform of a product. The post installation escalation rates varied from about 0.5% (for the R5.0 one-month measure) to 6% (for the R1.2 6 month measure) for this set of releases, and current releases, such as R3.1, R4.0 and R5.0 have significantly better quality than earlier releases, such as R1.1 and R1.2 of the product. A series of dots above a bar indicates a statistically significant improvement from the previous release.

Our primary data source for analysis of customer perceived software quality is the Avaya corporate customer defect tracking systems. The issues of data quality for IQ are described in detail in (Mockus and Weiss 2008).

4.4 Analysis of Productivity

Questions: What is developer productivity? How is it changing over time?

To measure developer productivity we follow the approach introduced in (Mockus et al. 2002). We first select a subset of developers who together contributed 80% of the changes each year. We refer to them as the core group. The number of changes to the source code by

this core group is then divided by the group's size to get the productivity measured by number of changes per developer per year. We consider only changes made to the source code files³.

Most mature development groups have practices that require or strongly encourage developers to keep their code in a version control system. Individual developers benefit because others are aware of their changes, and we did not observe any developers that would avoid keeping their code in the project's version control system. We count check-ins of individual files (delta) as separate changes even though they may be made under the single commit or MR.

We present productivity trends for a sample project in Figs. 5 and 6.

In addition to productivity, we show the growth in the codebase for that product and the churn of the core group of developers. Not surprisingly, the charts indicate that increased churn tends to lead to decreased productivity.

Figure 5 shows a rapid drop and recovery of developer productivity between 2005 and 2009. After a breathtaking rise, the developer numbers fell precipitously starting in 2003. Figure 6 indicates the distribution of developers by their experience on the project. As the product matured, the percentage of developers that stayed with the project for more than three years increased from 25% in 2003 to 70% in 2006, when the project was offshored and most of the experienced developers left or transferred. Of note, the productivity started dropping well before offshoring, possibly indicating less need to develop new features and, possibly, causing the initial drop of developer population.

Our primary data sources for productivity analyses are Avaya project change management systems using the systems' automatic tracking of changes to developer login. The main sources of uncertainty are the possible changes of developer logins and the imperfect reflection of developer productivity provided by number of changes per month.

4.5 Analysis of Predictability

Question: How well do Avaya projects predict their completion dates?

Avaya employs a business process with gated reviews to move projects forward and to synchronize such functions as development, training, documentation, services, installation, and customization. Depending on project characteristics, Avaya uses a variety of processes within the development phase, including iterative and traditional waterfall processes. Predictability of development is important to make sure all functional areas allocate resources appropriately.

We are able to extract planned and actual dates from a number of sources and compare them for consistency. Figure 7 shows how well projects predict their completion dates at different gates in the development cycle. The Y axis shows how late or early a project completion estimate is relative to its actual completion. The horizontal line at 100% indicates on-time completion. The distribution of estimates at different gates in the development cycle is shown as shaded ovals. For example, at the OK-to-Plan gate, which marks the end of the feasibility phase, the shaded oval shows the distribution of estimates. The median estimate is indicated by a small black horizontal oval within the larger oval. One can see that very few projects fulfill their estimates. The exact median is sensitive, so we do not provide its value. However, the chart is drawn to scale, so one can see that the

³ We use the same range of programming languages indicated in Section 4.2 to include in what we consider to be source code.

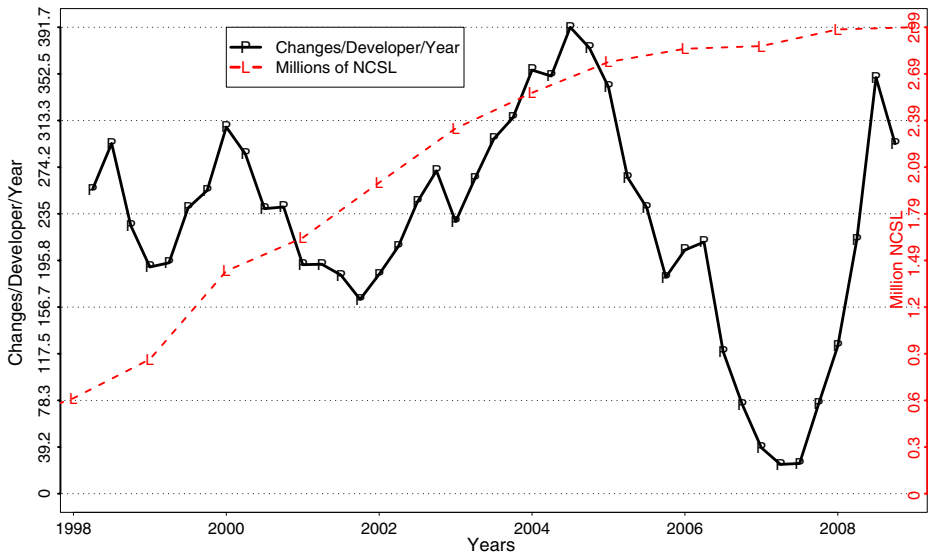


Fig. 5 Productivity trend for a sample product

median estimate is significantly different from 100%. Furthermore, even when development has advanced to a later gate, there is only a small improvement.

Figure 8 shows a relation between three factors and projects' ability to predict their schedule. For example one factor is the degree of cross-project and cross-division cooperation required. We found that projects involving cooperation among divisions were worse at predicting completion dates than projects that were located within a single division but had development staff at several different sites, which were worse than projects that were located at a single site. Without quantitative data an experienced project manager

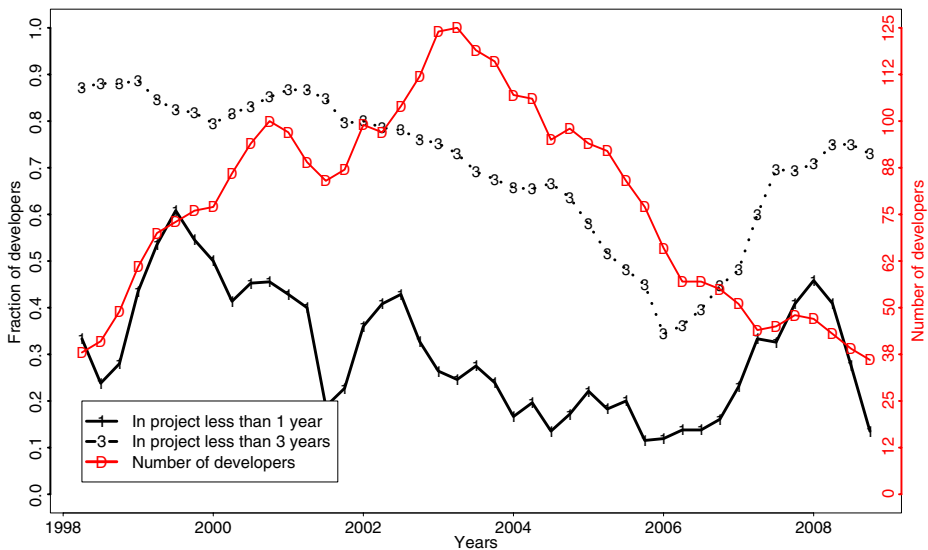


Fig. 6 Developer churn for a sample project

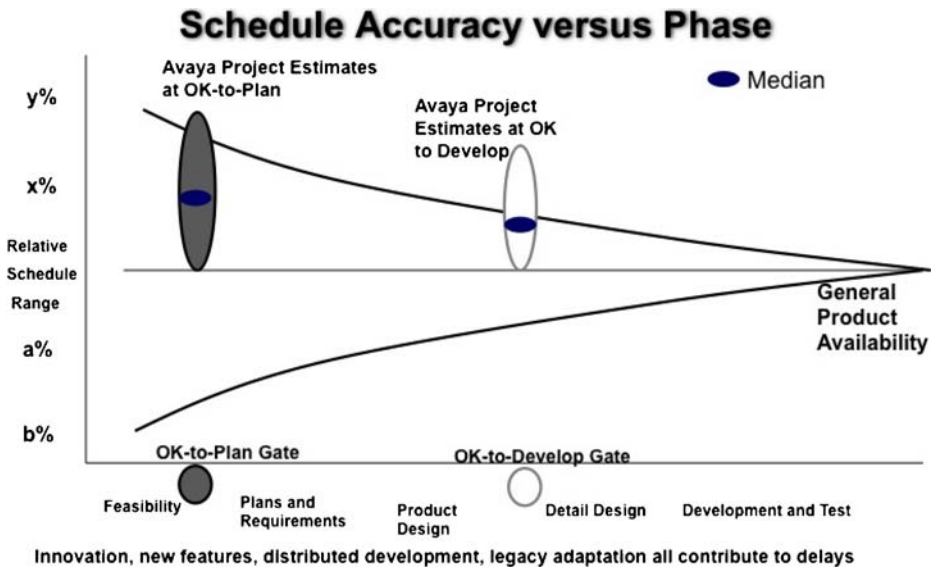


Fig. 7 Schedule predictions

might guess the same result, but would not know the magnitude of the difference. We also look at predictability by phase and predictability by variables such as project complexity or size, number of sites, or development methodology. We track the predictability trend to ensure that practices are being put in place to improve predictability. At the time the charts in Figs. 7 and 8 were generated we observed that Avaya was not good at prediction and that Avaya's ability did not improve as the development cycle progressed. Focusing attention on

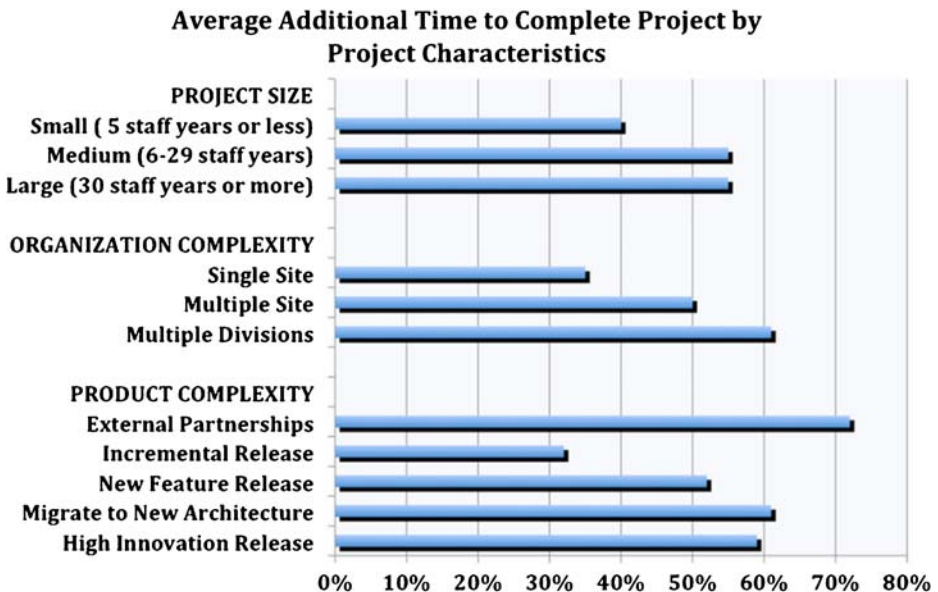


Fig. 8 Some factors that affected predictability (circa 2005)

this issue through quantitative analysis led to the use of better prediction techniques and subsequent analysis showed improvements. This is a good example of an improvement resulting from (quantitatively) highlighting a problem.

Our primary data sources for predictability analyses are the committed and actual dates that are tracked in Avaya's program management repository. This data is updated and tracked on a monthly basis and is about 95% accurate.

4.6 Domain Expertise

Question: How quickly can we train new developers to become effective?

Avaya provides new releases of its products on a regular basis, and continually develops new products using new technology. We use the term domain expertise to mean skills and knowledge that contribute to the product development and deployment process. Particularly, the product development and support community within the company must have the following knowledge and skills as part of their domain expertise.

- Product knowledge and structure, such as the internal workings of the software controlling a telecommunications switch.
- Specific technologies that are critical to Avaya communication products, such as telephony protocols, telephony call control interfaces, and application servers.
- How Avaya customers use our products and solutions, such as how call centers operate and how integrated management systems are deployed.
- How Avaya products work together in a user's domain to provide an overall business solution.
- What software development processes Avaya uses, and what are available for use, including quality-focused software practices, iterative development processes, debugging techniques, and testing techniques.

Because this is a critical and complex topic, the report takes time to examine all aspects of it. For purposes of brevity and simplicity, we will only discuss here the training of developers to become knowledgeable in the internal workings of Avaya products. Our measure for this is their productivity in terms of changes per developer per month, the same productivity measure as used in Section 4.4. Figure 9 shows learning curves for new developers as a function of the number of months since joining a project. The figure shows

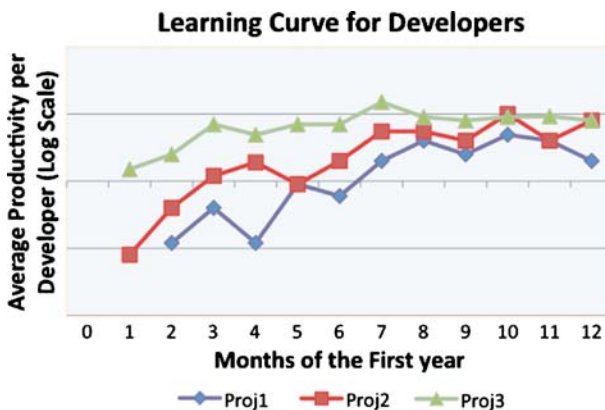


Fig. 9 New developer learning curves

curves for three different projects and all have similar behavior, reaching asymptotes after about seven months with a project. In the past we have observed similar curves reaching asymptotes in six months to two years (Mockus and Weiss 2001).

The rate of increase in productivity and the time to reach the same productivity as experienced developers vary with a number of factors, such as characteristics of product and project, training methods used, the number of sites across which work is distributed, and skill of the developers.

These curves have strong significance for development organizations that are trying to start new development sites or that have high turnover rates at sites. Aggregated over all new developers (with different training and skills) in a project, this example shows it takes an average of 7–8 months for them to approach the productivity of project veterans.

We have not done modeling to quantify the impact of individual factors. This will be a future area of work for us.

4.7 Transfer of Work Offshore

Questions: What is the relative productivity of offshore locations? How long does it take for a new offshore location to reach the same level of productivity as an established site? What are the key factors in bringing an offshore location to the same productivity level as an established location?

Avaya's R&D projects have migrated to lower cost offshore sites, such as India (see Fig. 10). A recent focus of our assessment has been to understand the impact of transfer of work offshore and to recommend practices to improve its effectiveness. The learning curves shown in Section 4.4 provide the start for an analysis of how to achieve highly productive offshore teams.

While early attempts at offshoring encountered technical problems, as data connections became better and experience with offshoring increased, the next hurdle to overcome was the dearth of software developers and managers with several decades of expertise. While college graduates in India and elsewhere could fill the entry-level positions, the explosive growth in offshoring has exposed a lack of senior developers and architects needed to train and mentor the newcomers to the project and perform tasks such as creating and maintaining architectures.

Therefore, as the experience of the offshore locations increases, the tasks they can successfully accomplish change as well. Our objectives are to assess the expertise in the various software development sites, to quantify the rate at which new developers increase their productivity, and to understand and compare various practices used to offshore software projects.

In our 2008 assessment, we conducted interviews with 50 Avaya offshore project personnel and onshore developers and managers that were supporting offshore teams. Furthermore, we have collected project version control data to quantify productivity and other aspects.

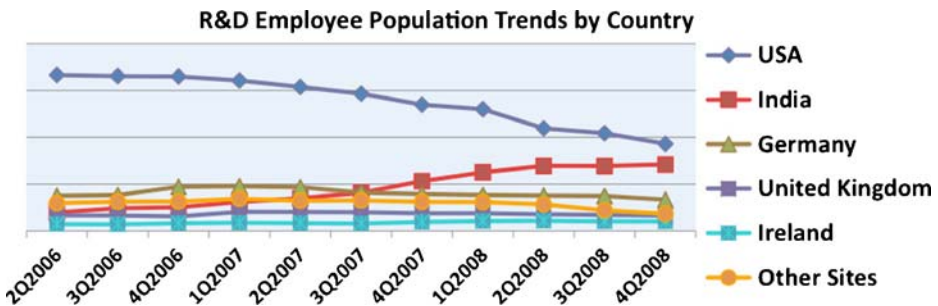


Fig. 10 R&D population trends by country (2006–2008)

The range of projects we investigated had used offshoring in a variety of ways. Some projects would offshore just one of the functions, for example testing or current engineering (fixing bugs on released versions of software). Other projects have offshored new releases, but were still developing new products in a traditional way. There were some projects that have offshored everything, including new product development.

The level of assistance the offshore teams got in the transition process varied as well. Some teams just received the code documentation and build environment and had little or no interaction with the original staff. Other teams went through an elaborate knowledge transfer process with the team handing over the product spending significant time preparing documentation and training the new team. Training activities included travel from and to the offshore location. The set of transferred artifacts also varied in richness from source code and build environment, to training courses, and intricate problem reporting systems that encoded details of how problems were solved in the past and that included navigation features that helped understand how the project worked in the past.

To answer the question about achieving productivity parity, we looked at the relationship between developer tenure with the project and productivity over time within the project, as seen in the productivity curves in Fig. 9. We were able to gather and analyze data from the configuration control systems that allowed us to determine, for each month since joining the project, the average developer productivity per month over the set of developers who had been with the project for that number of months. For example, for all developers who had been with the project for six months, we could determine their productivity for the six month period.

Fitting a regression model with the response as the logarithm of the number of changes and the predictors as developer identity and months since the developer joined the project we find that productivity increases for seven months and then appears to reach a plateau as shown in Fig. 11.

Figure 11 shows fitted coefficients with standard error bars for the first 12 months of a developer's participation in the project to illustrate the trends. The horizontal axis shows months since a developer joined Avaya and the vertical axis shows estimated productivity (in terms of number of changes per month) for developers of this tenure. The productivity is

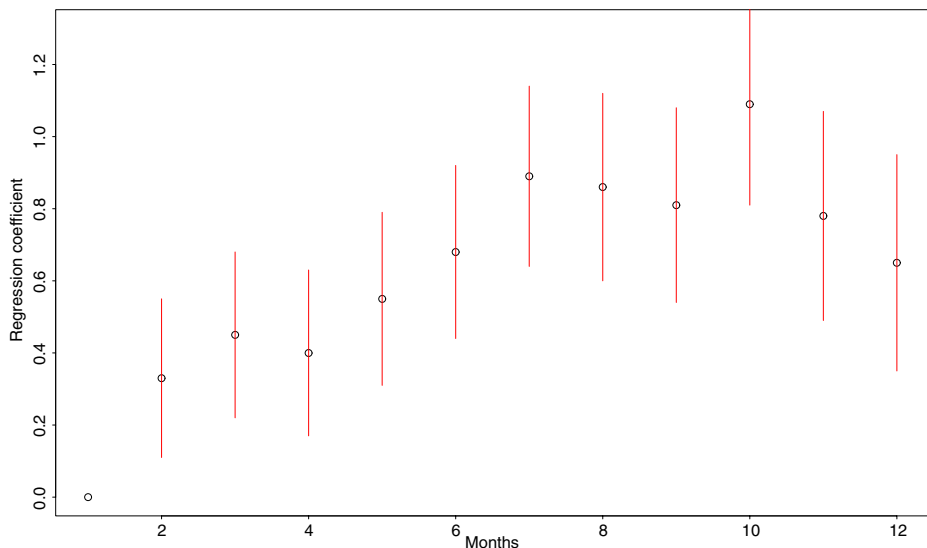


Fig. 11 Illustration of the estimated productivity increase over time

estimated using a regression model that also accounts for the past changes a developer completed before reaching that tenure. It is important to note two crucial limitations when interpreting the results. First, the plateau does not imply that developers are not getting any more productive. In fact, as shown in (Zhou 2009) the types of changes that experienced developers make are different than novice developers. In particular, experienced developers engage in more critical and complex tasks. Furthermore, mentoring and coordination are additional tasks that occupy more senior developers but are not directly reflected in source code changes. Therefore, the numbers of changes does not fully reflect the output of developers and has a different meaning for junior and senior developers because of the differences between the types of tasks they undertake.

In summary, we have found that effective transition of software development to offshore teams requires careful project selection, effective knowledge transfer programs, and the establishment of offshore teams with needed skills. In addition, it typically takes 6 to 10 months for a new developer to become an effective contributor who can handle complex changes in the code. Because of the proprietary nature of the information we cannot discuss the relative productivity of various AVAYA R&D locations.

5 Software Practices

Question: What software development practices are most often and most effectively used?

We assess the extent of usage and the effectiveness of the deployment of key software practice areas and individual practices that make up these areas. The key software practice areas were based on the practices needed to meet stated Avaya goals, which changed over time, and our observation of industry practices. We have mapped the Avaya key practices to CMMI practices. The practice areas are:

- Customer focused development: Practices that emphasize customer input and feedback throughout the development lifecycle
- Design Quality In: Practices that provide a focus on quality early in the development lifecycle.
- Improve Testing: Practices that improve the automation and comprehensiveness of developer and system tests.
- Software Project Management: Practices to support planning, monitoring and controlling an individual software project.
- Multisite/offshore development: Practices that support the effective development of software across geographic boundaries, especially those involving offshore teams.
- Architecture-guided iterative development: Practices drawn from agile and traditional methodologies that are organized into a family of iterative processes, where development is guided by a well-defined architecture.
- Cross-project cooperation and coordination: Practices that support cooperation and coordination across individual project boundaries, which cooperation is required for platform-based development and solution-based development.

Our assessment for each practice is based on the following factors.

- We have established a set of criteria for effective deployment of the practice based on industry and Avaya good practices. See Sections 5.1 and 6.1.1 for examples of criteria.
- We assess the practice relative to the criteria using analyses such as those described in Section 4.

- We determine the extent of usage from our input sessions and our partnerships during the year with Avaya projects.
- We ask respondents in our input sessions for their assessment of the effectiveness of the practices in their organization and the reasons for that assessment. Figure 12 is an example box plot of the range of reported effectiveness based on these interviews. The reader should keep in mind that these subjective assessments are by experienced engineers.

Our overall assessment is a judgment based on these factors.

5.1 Example Practice Area—Design Quality In

The following 11 individual practices are representative individual practices associated with the “Design Quality In” practice area.

- Baseline requirements and place them under change control.
- Specify and update an architecture sufficient to guide development.
- Deploy internal technical standards.
- Create well-defined interface specifications (Clements et al. 1983).
- Perform cross-division architecture reviews (Maranzano et al. 2005).
- Establish component integration and reuse program.
- Review requirements, interface specifications, design artifacts, and test scripts.
- Perform inspections of new or changed code.
- Perform automated build management (at least daily) with automated sanity tests.
- Carefully manage 3rd party deliverables and dependencies on other projects.
- Refactor a criteria-based selection of modules.

Each of these practices may be further partitioned into sub-practices. For example, in Section 6.1.1, we identify six key sub-practices of the automated build management practice.

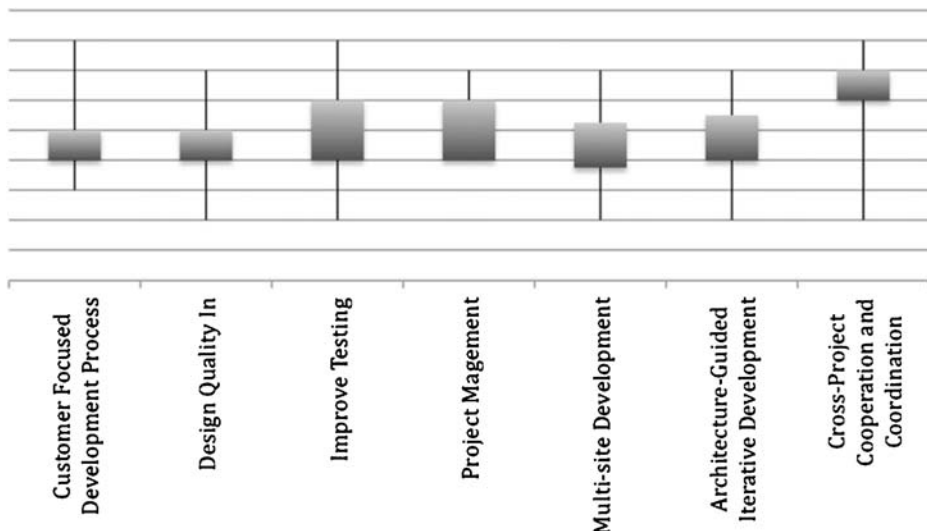


Fig. 12 Range of reported effectiveness (2nd and 3rd quartiles) for selected software practices

We have defined criteria for effective deployment of each of these practices. For example, the criteria for effective deployment of “manage 3rd party deliverables” are the following.

- All 3rd party deliverables are identified.
- Quality policies are in place and communicated to 3rd party owners.
- Policies are in place on conditions for accepting updates.
- An acceptance test program is in place for each 3rd party deliverable.
- A development team member is identified as “local owner” of each 3rd party deliverable.
- Schedule and quality impact of 3rd party deliverables is assessed.

Our recent assessment of the individual practices for “Design Quality In” is shown in Fig. 13. In the figure the X-axis represents how widely the activity is deployed (Usage), and the Y-axis represents how effectively the process has been deployed (Effectiveness) based on criteria for each practice. For example, automated build management is widely deployed and Avaya R&D projects are highly effective in using it (see Section 6.1.1). On the other hand, there is limited usage of structured refactoring techniques (Fowler 1999) and where deployed they are deployed with medium effectiveness. The Overall circle represents our judgment of the total set of “Design Quality In” practices, which is medium to high usage and medium effectiveness.

These charts provide guidance by identifying software practices to target for improvement. Avaya provides guidance on best practices to projects, but leaves the implementation of each specific practice to each project.

Figure 14 shows our overall assessment of design quality in from 2002 to 2008. This practice area shows initial improvement, then decline followed by slow improvement in usage and minimal improvement in effectiveness. Avaya has inherited a long tradition from Bell Labs of quality products and quality-focused development processes (see (Bell Labs Technical 2000), for example), and quality remains a strong emphasis for Avaya R&D leaders and staff.

5.2 Example Individual Practice—Improve Predictability

We define predictability as the ratio of the actual length of schedule to the committed length of schedule. In Avaya, a schedule commitment is made at a formal business gate when the

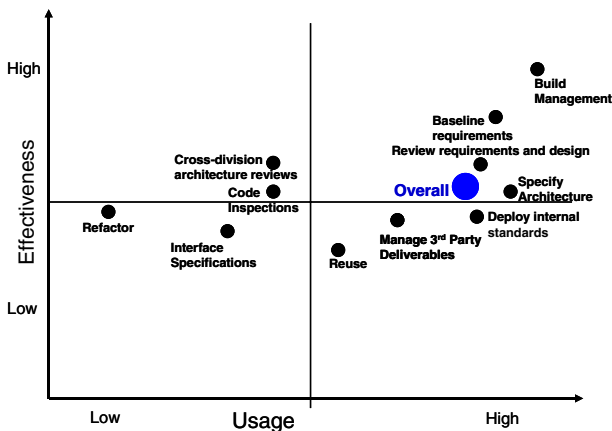


Fig. 13 Assessment of selected individual practices for “Design Quality In”

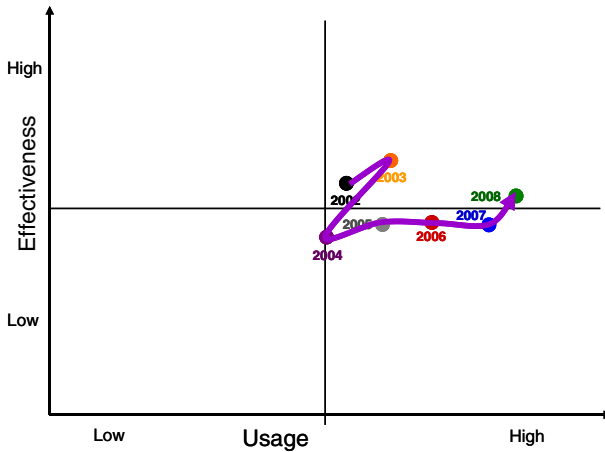


Fig. 14 Multi-year trend: design quality in (2002–2008)

requirements and architecture are baselined, a development plan is in place and resources are ready to be allocated to the project.

“Improve predictability” is an individual practice in the project management software practice area that includes prioritization of release content, disciplined and repeatable estimation, proactive program change management, and automation of build and test processes.

The following are example criteria for effective deployment of the “improve predictability” practice.

- Feature content is prioritized with a cut-off point negotiated among product management, development and other stakeholders.
- Project commitments are based on expert estimates, experience of previous projects and committed staff levels.
- Program change management is performed proactively and guided by a defined set of performance metrics.
- There is a project culture of automation for daily or continuous builds, sanity tests, unit tests and regression tests.
- The release is predictable. The actual deployment date is within 30 days of the committed deployment date.

As discussed in Section 4.5, predictability was a challenge for Avaya projects. As shown in Fig. 15, the effectiveness of predictability worsened from 2003 to 2005 and then improved in recent years.

6 Assessment Follow-up: Recommendations and Impact

We create a set of five to seven prioritized recommendations each year based on the assessment findings and identify a suggested owner for each recommendation. Suggested owners typically include the following roles.

- The project team lead if the recommendation is within the scope of an individual project,

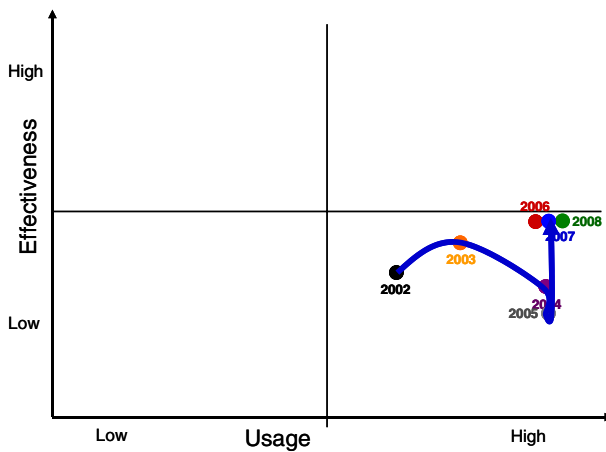


Fig. 15 Multi-year trend: predictability (2002–2008)

- The R&D leader for an organization if the recommendation is within the scope of an individual R&D organization,
- The product management leader within a division if the recommendation requires the leadership of the product management team,
- The division general manager if the recommendation requires cooperation with a division across the entire functional areas of the business, or
- Avaya's operations leader if the recommendation requires cross division cooperation.

No individual role is assigned more than two or three areas of responsibility in any given year.

The report provides an abstraction of each recommendation in a summary table similar to Table 1, and a detailed description in the body of the report.

The first column of the summary table contains the recommendation abstract. The second column identifies the software practice addressed by the recommendation, and the third column identifies the suggested owner.

Two example recommendations are described in Section 6.1. Section 6.2 describes how the recommendations are deployed.

Table 1 Example recommendation summary table (excerpt)

Recommendation	Software practice area addressed	Deployment responsibility
1. Recommendation One: Continue the software improvement program with a concentration on customer-focused development Monitor the impact of the program	• Customer Focused Development	• Division Product Management leader
2. Recommendation Two: Improve build management practices as a lever to improve quality. Monitor the impact.	• Design Quality In	• Project team lead for each project
3. Recommendation Three: Improve the program for transfer ... Monitor the impact.	•....	• R&D Vice President for each division program

6.1 Example Recommendations

We have made about 30 separate recommendations between 2002 and 2008 covering a variety of topics, such as the following.

- Front end planning,
- Deployment of architecture-guided iterative and agile practices,
- Software project management,
- Multi-site and offshore development practices, and
- Domain expertise of the R&D staff.

The following sections describe two example recommendations. The automated build recommendation is an early recommendation, initially made in 2003. The customer-focused development recommendation is more recent, initially made in 2006.

6.1.1 Automated Build Management Recommendation

We classify automated build management as a practice in the design quality in practice area (see Section 5.1).

Avaya had several challenges in build management when the corporation was initially formed. While some Avaya projects had very sophisticated build management practices that supported distributed development and automated sanity testing⁴, the practices were not standard across the corporation. Build practices in many projects were inefficient, not fully automated and limited in their testing and reporting capabilities. In many Avaya projects, build management practices were more of a hindrance than a help to developers in rapidly integrating and testing their code.

Our recommendation was to improve build management practices as a lever to improve quality including the following set of industry-proven good practices (Appleton and Berczuk 2003; McConnel 1996).

- Good Practice 1. Treat load check-in as a quality gate
- Good Practice 2. Automatically track versions of all components of a load
- Good Practice 3. Establish a fully automated build and sanity test process.
- Good Practice 4. Perform daily builds and sanity test
- Good Practice 5. Automatically link change tracking and version control.
- Good Practice 6. Define and track load building metrics

We provided details on each practice in a separate report and identified example Avaya projects who could serve as mentors to other projects.

6.1.2 Customer-Focused Development Recommendation

Customer satisfaction is critical to Avaya's business and achieving high customer satisfaction metrics is a primary Avaya business goal. We recommended deploying *customer-focused* software development practices, one of the software practice areas described in Section 5, as a means to achieving this goal. Our recommendation includes

⁴Sanity testing is the practice of running a few tests after a build to ensure that the product was built correctly and that basic functionality works.

making customer-focused practices such as the following standard throughout the development lifecycle.

- Conduct joint customer visits by product managers and R&D technical leads to understand customer needs and operations. Establish a “walk in the shoes” program for architects and other lead technical staff to better understand customer needs and customer pain.
- Establish customer advisory panels to provide input and periodic feedback, especially for complex product portfolios with a diverse customer base.
- Extend the root cause analysis of customer found defects to establish programs in all organizations to identify and deploy corrective actions based on root cause analysis trends.

As with the build management recommendation, we identified Avaya projects that are deploying each of these practices.

6.2 Deployment of Recommendations

Recommendations are deployed in partnership with business and R&D leaders. We conduct follow-up sessions with individual business leaders, R&D leaders and product management leaders of each organization. The sessions are typically structured based on the following list of questions for each recommendation where the individual has a suggested role of responsibility.

- Is the recommendation relevant to your organization/team?
- If relevant, is it already deployed in your organization/team?
- If relevant and not deployed, what are the barriers to deployment?
- What actions are you prepared to take?
- How can our team help in deployment?

Note that as a result of these sessions, a recommendation may not be deployed in a particular organization either because it is not relevant to that organization or the barriers for deployment are too large. In addition, the recommendation may be adapted to make it more deployable in a particular organization. When appropriate actions for an organization are identified for a recommendation, action plans are then defined, deployed and tracked. Results are incorporated into the next year’s assessment report.

6.3 Impact of the Assessments

We evaluate the impact of the assessments based on the usage and effectiveness of the software practices areas, in particular those that we target for improvement. The extent of usage and effectiveness of each software practice are assessed as described in Section 5. The trend over time serves as a measure of the impact of the assessment for that practice.

For example, our assessment in 2002 and 2003 was that Avaya projects had medium to high usage of automated build management practices with low to medium effectiveness⁵ as shown in Fig. 16. In 2003 and 2004, we made specific improvement recommendations as described in Section 6.1.1. These recommendations helped create a focus in Avaya R&D in 2004 and 2005 to improve build management practices. The good practices in build management described in Section 6.1.1 are now widely deployed and build management is an effective, efficient practice for most Avaya projects (see Fig. 15 for 2006, 2007, and 2008).

⁵ The scales for Usage and Effectiveness are defined the same as in Section 5.

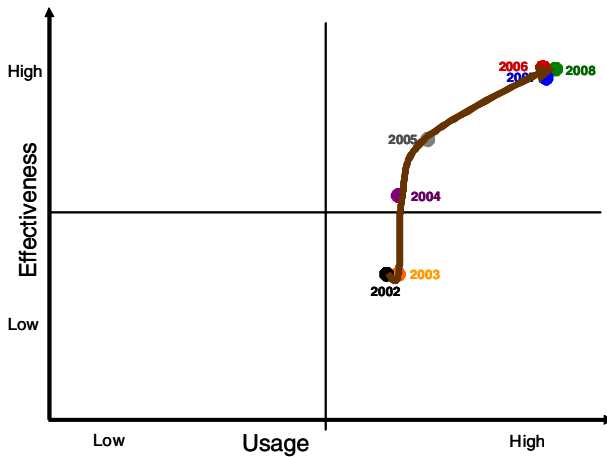


Fig. 16 Multi-year trend of usage and effectiveness of automated build management practices (2002–2008)

Another example is customer-focused development as described in Section 6.1.2. Our assessment in 2004 and 2005 was that Avaya projects had low usage of customer focused development practices with medium to low effectiveness of those practices where they were deployed. Note the placement for those years in the lower left hand quadrant of Fig. 17.

Many Avaya projects have begun deploying customer-focused development practices. For example about 25% of Avaya projects are using iterative or agile practices that include regular opportunities for customer feedback. By 2008 the usage of customer focused development practices has increased to medium usage and the effectiveness has increased to a medium-high rating. Nevertheless, like many other companies, Avaya needs to improve in the deployment of customer-focused development practices.

Both of these examples show year-by-year improvement after a recommendation is made in our annual assessment, but that is not always the case (see Fig. 14 for example). Figure 18 shows the trend curve for several practices, including customer-focused development and automated build management. A trend curve is a curve drawn connecting

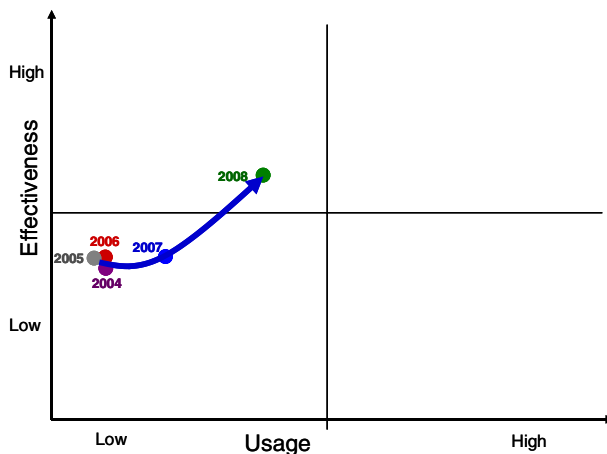


Fig. 17 Multi-year trend of usage and effectiveness of customer focused development practices (2002–2008)

the 2002 assessment of the practice sequentially through the 2008 assessment of the practice. Some observations about the trends follow.

- Both Automated Build Management and Customer-Focused Development show year-over-year improvements in usage and effectiveness in the initial years after a recommendation is made. In both cases, Avaya organizations took actions (as described in Sections 6.1.1 and 6.1.2) to improve the effectiveness of those practices. Automated Build management is widely deployed, but customer focused development is not widely deployed in part because of the organizational layers between the R&D teams and end customers.
- Predictability shows initial improvement then decline followed again by improvement in effectiveness. Predictability is widely deployed and now very effective in some, but not all, parts of the company. Predictability has been ineffective especially for projects with large dependencies on other organizations for key components in their project.
- Test Automation increased quickly in effectiveness, but not in usage. Where deployed the practice has been highly effective, but it is a struggle to deploy the practice widely within Avaya. Test Automation requires an initial investment in staff resources for infrastructure and automation that is difficult to allocate with tight development resources.

It is important to note that our assessments are aimed at improving software development in Avaya's software development organizations, and are not intended to be a tool to evaluate individual developers or managers. Organizationally, the ARC is part of Avaya Labs Research, which is not part of any Avaya software development organization, so there is no incentive to bias the report in favor of one organization or another. As with any set of published data, it is possible for the data to be used for purposes other than its original intent, for example as a means for evaluating individuals within organizations, or for comparisons with other companies or other organizations. Another danger is that individuals might attempt to bias the data that we collect in order to improve the performance of their organization in the eyes of our assessment. There are several guards against misuse and distortion of the data, as follows.

- We validate the assessment by comparing subjective and quantitative views of the data, and by occasional random samplings of parts of the data.

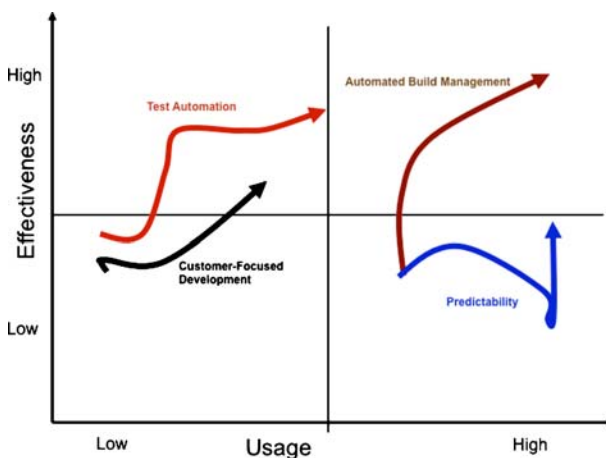


Fig. 18 Multi-year trend of selected practices showing different behaviors (2002–2008)

- The data are gathered from a variety of sources and represent measures that would be very difficult to optimize together. There is no single measure to optimize. In addition, some of the data are provided by organizations external to the development organizations. Customer data on which interval quality is based is one example; it is collected and analyzed from reports filed by customers.
- Once an assessment is concluded, it is widely published within Avaya and we discuss the findings and recommendations with a variety of developers, managers, and executives within the company.
- We specifically explain the rationale and approach (GQM) that we use in the assessments, in discussions with Avaya executives and managers to guard against miscomparisons. The reports are proprietary and are not published outside the company so that external organizations do not have access to them.

Finally, in the seven years in which we have been producing the assessments, we have seen no evidence of any attempts to bias the data.

7 Conclusions

The Avaya State of Software Reports are but one example of process improvement through monitoring and feedback. See, for example (Basili et al. 1992). Because our reports focus on specific business goals and collect consistent data across the entire company, we can see clear trends year by year, and can make and observe the effects of specific recommendations. We believe that our approach tends to solve the problems that many organizations have in establishing and maintaining measurement programs, such as those described in (Brown and Goldenson 2004), and can be used as a model for those wishing to gain insight into how well their software development organizations function and how to improve their functioning. In addition, the reports often identify problems that are appropriate for our research organization to solve, allowing us to operate according to the approach suggested in (Stokes 1997). In summary, we believe that our approach has the following advantages.

- Consistent data allows us to identify year-over-year trends.
- Regular analysis, which allows for publication of an annual report, allows us to track the effects of recommendations and to identify budding problems.
- Problems difficult to solve in a development organization can be assigned to a research organization. Solutions, when transferred back to the development organizations, can be observed for effectiveness, allowing the establishment of feedback loops between development and research for refinement of solutions.
- Data collection, analysis, and validation is performed by a stable group of people experienced with product development problems, knowledgeable about and trusted by the business units, thereby lending confidence in the results.
- Validation of results with development organizations creates credibility for the report and willingness to implement recommendations, thereby leading to improvements appreciated by all organizations.

There is, of course, an initial investment required to establish a stable group of people with the experience and skills needed to produce the annual report. The core of the ARC, our group that produces the report every year, consists of two people who have been

involved in the report's production every year for seven years. Although producing the report every year is an important part of their efforts, it is only one subset of a larger set of their activities aimed at improving the way that software development is done in Avaya. In producing the report they are also supported by developers and researchers who provide specialized skills as needed.

Collection of data used for the report is continuous, and for much of the quantified data automated, but development of the report is started at the beginning of the last quarter of the year and publication is in early January. Maintaining a regular cycle helps in trend tracking, in resource allocation for implementing recommendations, and in getting the attention of developers, managers, and executives throughout the company, who know to expect the report at the beginning of every calendar year.

7.1 Next Steps

We will continue to prepare an annual state of software report for Avaya and expect to focus on the following areas in the coming years.

- As shown in Fig. 8, predictability can be correlated with other factors. The chart provided insight into how to improve predictability, and we will examine these factors in our future work.
- We have recently begun to analyze the effect of agile development in a large enterprise with real time complex products and distributed development on measures such as predictability, time to market, and product quality.
- We will improve our ability to estimate the accuracy of our data and analyses.

We are also interested in benchmarking with other companies who are performing similar analyses and with whom we could assure data comparability.

It is an open question as to how the various factors analyzed interact with each other, and is a subject of future research.

The techniques we use in producing the report could be applied in any organization that can define its goals, that is willing to collect the data needed to measure progress towards those goals, and that is willing to make an investment in improving its development practices. Although we typically think of such organizations as companies, or business units within companies, the techniques could be applied to any entity that develops software. For example, a worthwhile goal might be to produce a report on the state of software in the U.S. Certainly, the challenges would be large, but the payoffs in terms of knowing where one should invest to improve the way software is developed in the country would be large as well.

Acknowledgements We thank the 200+ people who have contributed data, participated in interviews, validated results, and helped with the analysis of data for the State of Software in Avaya reports.

We especially thank David Bennett, Neil Harrison, Joe Maranzano, and John Payseur for their contributions to early assessments, Trung Dinh-Trong for his contributions to recent reports especially in the areas of test automation and code coverage, Zhou Minghui for her recent work in Avaya on developer productivity, Evelyn Moritz for her analyses and feedback to the report, Pierre Osborne for his leadership in establishing and supporting Avaya's R&D data warehouse, and Jon Bentley for his careful and helpful reviews of the reports. Thanks to Ravi Sethi for his executive sponsorship and support of the ARC program and Christian Von Reventlow for his enthusiastic assistance in establishing deployment programs in partnership with business and R&D leaders.

Appendix

Questions Used For Input Sessions

As one source of input we use individual and small group input sessions with R&D and product management. These sessions cover all divisions, major R&D sites, various levels of technical staff and management, and a variety of R&D roles, including architects, developers, project and program managers, testers, documentation, and support. Our findings are reviewed with them and adapted based on their feedback. Each session is based on two sets of questions, those used in each session, and a set of software practices that we tailor based on the role of those in the input session. Common questions include:

- What are the 3 primary software-related areas that your project is doing particularly well?
- What are the 3 primary software-related issues facing your project?
- What is your assessment of the software quality for your project and why?
- What is your assessment of the productivity for your project and why?
- What is your assessment of the domain expertise of your staff? Why?
- What software development approach(es) are used in your project?
- If you could change one software related area in Avaya, what would you change?

The nature of these sessions and the questions that we ask has evolved over time. For early reports we had less data on trends and on a variety of factors such as productivity, cost, and quality. Accordingly, we spent more time in individual discussion sessions. Currently, we are able to show session participants charts and graphs showing trends and, as necessary, data about their organizations, and use those to focus the session. As a result, sessions are more to the point and briefer.

References

- Akao, Y. (Ed.) (1990) *Quality Function Deployment: Integrating Customer Requirements into Product Design*, Productivity Press, Portland, OR
- Appleton B, Berczuk SP (2003) *Software configuration management patterns*. Addison Wesley, Boston
- Basili V, Weiss D (1984) A methodology for collecting valid software engineering data. *IEEE Trans Softw Eng* 10:728–738
- Basili V, Weiss D (1985) Evaluating software development by analysis of changes: some data from the software engineering laboratory. *IEEE Trans Softw Eng*, SE-11(2):157–168, Feb. 1985
- Basili V, Caldiera G, Frank McGarry F, Pajeraki R, Gerald Page G, Waligora S (1992) The software engineering laboratory—an operational software experience factory. *Int Conf Softw Eng*, pp 370–381, May 11–15, 1992 ISBN: 0-8791-504-6
- Bell Labs Technical Journal, Special Issue Software, Volume 5, Issue 2 (April/June 2000), available at <http://www3.interscience.wiley.com/journal/97518354/issue>
- Brown M, Goldenson D (2004) Measurement and analysis: what can and does go wrong?, *Proceedings of the 10th International Symposium on Software Metrics (METRICS'04)*, September, 2004
- Chang H-F, Mockus A (2008) Evaluation of source code copy detection methods on freebsd. In: *Mining software repositories*. ACM Press, May 10–11, 2008
- Clements P, Britton K, Parnas DL, Weiss D (1983) *Interface specifications for the SCR (A-7E) extended computer module*, NRL Report 4843, Naval Research Laboratory, Washington, D.C., January 1983

- Fowler M (1999) Refactoring: improving the design of existing code. Addison Wesley, Boston
- Grady R, Caswell E (1987) Software metrics. Prentice-Hall, Englewood Cliffs
- Maranzano J, Rozsypal S, Warnken G, Weiss D, Wirth P, Zimmerman A (2005) Architecture reviews: practice and experience. *IEEE Softw* 22:34–43
- McConnel S (1996) Rapid development. Microsoft Press, Redmond, Washington
- Mockus A (2007) Large-scale code reuse in open source software. In: ICSE'07 Intl. Workshop on Emerging Trends in FLOSS Research and Development, Minneapolis, Minnesota, May 21 2007
- Mockus A, Weiss D (2001) Globalization by chunking: a quantitative approach. *IEEE Softw* 18(2):30–37
- Mockus A, Weiss D (2008) Interval quality: relating customer-perceived quality to process quality. In: 2008 International Conference on Software Engineering. ACM Press, Leipzig, Germany, pp 733–740, May 10–18 2008
- Mockus A, Fielding RT, Herbsleb J (2002) Two case studies of open source software development: apache and mozilla. *ACM Trans Softw Eng Methodol* 11(3):1–38
- Rifkin S (2001) Why software process innovations are not adopted. *IEEE Softw* 18:110–112
- Stokes D (1997) Pasteur's quadrant: basic science and technological innovation. Brookings Institution, Washington
- van Solingen R, Berghout E (1999) The goal/question/metric method: a practical guide for quality improvement of software development. McGraw-Hill, New York
- Walston CE, Felix CP (1977) A method of programming measurement and estimation. *IBM System Journal* 16(1):54–73
- Weiss D, Bennett D, Payseur J, Zhang P, Tendick P (2002) Goal-oriented software assessment. International Conference on Software Engineering, May 2002
- Zhou M (2009) Learning legacy products: how product structure shapes organization, ICSE workshop on Socio-technical Congruence, May 19, 2009



Randy L. Hackbarth coordinates a team dedicated to improving the state of the practice of software development in Avaya. Before joining Avaya, Randy worked for 20 years in Bell Labs at Lucent Technologies and AT&T with a focus on establishing, coordinating and contributing to business unit - research partnership projects. Randy was also a founding member, interim CEO and business development VP of PaceLine Technologies, LLC - a Lucent Venture. His education background includes an MS in Computer Science and an MA in Mathematics, both from the University of Wisconsin-Madison. Randy is a member of IEEE and ACM. He can be reached at randyh@avaya.com.



Audris Mockus conducts research of complex dynamic systems. He designs data mining methods to summarize and augment the system evolution data, interactive visualization techniques to inspect, present, and control the systems, and statistical models and optimization techniques to understand the systems. Audris Mockus received B.S. and M.S. in Applied Mathematics from Moscow Institute of Physics and Technology in 1988. In 1991 he received M.S. and in 1994 he received Ph.D. in Statistics from Carnegie Mellon University. He works at Avaya Labs Research. Previously he worked at Software Production Research Department of Bell Labs.



John Douglas Palframan is a research scientist with Avaya labs. He works with Randy on improving software practices in Avaya. Before joining Avaya, John was a technical manager in Bell Labs responsible for developing communications software and software development tools. He has organized annual software development conferences for the company since 1992. John is on the executive committee for the NJ Coast chapter of the IEEE. His education background includes an M.Math and a B.Math (co-op) from the University of Waterloo. John can be reached at palframan@avaya.com.



David M. Weiss received the B.S. degree in Mathematics in 1964 from Union College, and the M.S. in Computer Science in 1974 and the Ph.D. in Computer Science in 1981 from the University of Maryland. He is currently the Lanh and Oahn Nguyen professor of software engineering at Iowa State University. Previously, he was the Director of the Software Technology Research Department at Avaya Laboratories, where he worked on the problem of how to improve the effectiveness of software development in general and of Avaya's software development processes in particular. To focus on the latter problem, he formed and led the Avaya Resource Center for Software Technology. Dr. Weiss's principal research interests are in the area of software engineering, particularly in software development processes and methodologies, software design, and software measurement. His best known work is the goal-question-metric approach to software measurement, his work on the modular structure of software systems, and his work in software product-line engineering.